Climate Hazard Trend Analyzer

Project documentation and deployment handover · October 12, 2026

Live application	https://climate-hazard-web.onrender.com
Live API	https://climate-hazard-api-7jyx.onrender.com (health: `/api/health/`)
Source code	github.com/MegharajPN/Climate-Hazard-Trend-Analyser-Using-React-Python, branch `main`
Database	TiDB Cloud Serverless (MySQL-compatible), cluster "Climate-Hazard", database `climate_hazards`
Hosting	Render (free plan): one Python web service and one static site, defined in `render.yaml`
Status	Deployed and working. Health check, city search, and heatwave analysis verified on the live site.

> **Credentials are not in this document.** The database username and password, and the Django secret key, are stored only in the Render dashboard and TiDB Cloud. Ask the project owner for access. A database password was shown in a screenshot during setup; it must be rotated (see section 11).

1. What the application does

The application analyses how climate hazards have changed for a chosen place over a chosen period. A user selects a location (city search, map click, or coordinates), a hazard, and a year range. The backend fetches historical daily weather from Open-Meteo, caches it in MySQL, detects hazard events with a stated definition, computes yearly counts and trend statistics, and returns plain-language insights and chart data.

Implemented hazards:

- **Heatwave** (default): daily maximum temperature above the 95th percentile for that time of year (baseline 1981–2010), in runs of at least 3 days.
- **Heavy rainfall**: days with precipitation above the 95th percentile of baseline wet days (≥ 1 mm).
- **Drought** (dry spells): runs of at least 10 consecutive days with less than 1 mm of precipitation.

Regions: a single point, or a box analysed as the mean of a 3×3 grid of points.

2. Architecture

```
Browser
  |
  |  https://climate-hazard-web.onrender.com        (Render static site, React + Vite)
  |        |
  |        |  API calls, base set by VITE_API_BASE
  |        v
  |  https://climate-hazard-api-7jyx.onrender.com   (Render web service, Django + gunicorn)
  |        |                          |
  |        |                          +--> TiDB Cloud (MySQL protocol, TLS)
  |        |                               tables: daily weather cache, fetched ranges, analysis
results
  |        +--> Open-Meteo archive and geocoding APIs (public, no key)
```

3. Cloud services and configuration

Service	Plan	Purpose	Where it is configured
GitHub	Free	Source repository; Render deploys from it	Repository settings
Render: `climate-hazard-api`	Free web service	Django API served by gunicorn; runs migrations at start	Render dashboard > service > Environment
Render: `climate-hazard-web`	Free static site	React frontend built with `npm run build`	Render dashboard > service > Environment
TiDB Cloud: Serverless cluster	Starter (free)	MySQL-compatible database, AWS ap-northeast-1 (Tokyo)	TiDB Cloud console
Open-Meteo	Free, no key	Historical weather archive and place-name search	Hard-coded URLs in settings

Environment variables (names and purpose; values held in Render)

Variable	Service	Purpose
`PYTHON_VERSION`	API	Python 3.11.9 for the build
`DJANGO_DEBUG`	API	Set to `0` in production
`DJANGO_SECRET_KEY`	API	Generated by Render; startup refuses the development key when debug is off
`DJANGO_ALLOWED_HOSTS`	API	Blank; Render's public hostname is added automatically
`CORS_ALLOWED_ORIGINS`	API	Must be `https://climate-hazard-web.onrender.com`
`DB_CONNECTION`	API	Must be `mysql`
`DB_HOST`	API	TiDB gateway host (section 4)
`DB_PORT`	API	`4000`
`DB_DATABASE`	API	`climate_hazards`
`DB_USERNAME`, `DB_PASSWORD`	API	TiDB credentials (secret; not recorded here)
`DB_SSL`	API	`1`: TLS is required by TiDB and verified against the system certificate store
`VITE_API_BASE`	Web	Must be `https://climate-hazard-api-7jyx.onrender.com`. It is compiled into the frontend, so changing it requires a redeploy.

Database connection details (non-secret)

Item	Value
Host	`gateway01.ap-northeast-1.prod.aws.tidbcloud.com`
Port	`4000`
Database	`climate_hazards` (utf8mb4)
Protocol	MySQL, TLS required
Username and password	Held by the project owner; not in this document

4. Implementation summary

Data ingestion and caching

- Daily maximum and minimum temperature (°C) and precipitation (mm) from the Open-Meteo archive, timezone GMT.
- Three cache layers: `DailyWeather` (one row per location and day), `FetchedRange` (what has been downloaded), and `AnalysisResult` (result documents, reused for 24 hours).
- Locations are rounded to 0.01°. A box is a 3×3 grid of points, averaged per day.
- Upstream rate limits (HTTP 429) and transient server errors are retried up to three times with backoff.

Data cleaning

- Temperatures outside −60 to 60 °C, and negative precipitation, are treated as missing.
- Missing days are never filled. A missing day breaks a run of hazard days.
- Years with less than 80% daily coverage are shown but excluded from counts and trends.

Trend statistics

- Change over the period: mean of the first five valid years compared with the last five.
- Ordinary least squares slope (per decade) and R^2.
- Mann–Kendall test with tie correction, and Sen's slope.
- Five-year rolling mean of counts.

API endpoints

Path	Purpose
`GET /api/health/`	Health check (returns `{"status":"ok"}`)
`GET /api/geocode/?q=`	Place-name search (2–100 characters); cached 24 hours
`GET /api/analysis/`	Hazard analysis. Parameters: `lat`, `lon`, optional `box_size`, `hazard`, `start_year`, `end_year`, `percentile`, `min_days`, `baseline_start`, `baseline_end`, `label`
`GET /api/analysis/export/`	Same parameters; yearly results as CSV

Validation and limits

- Latitude and longitude ranges; years from 1940 to the current year; start before end; maximum span 80 years.
- Baseline at least 10 years. Hazard values restricted to an allow-list.
- Anonymous limit 60 requests per hour; analysis endpoint 20 per hour (HTTP 429 when exceeded).
- Database access through the ORM only.

Frontend

- Region picker: city search, latitude/longitude, or box, with a Leaflet map.
- Hazard selector (heatwave by default) and year range; advanced settings for thresholds.
- Results: KPI tiles, insights, count and duration charts, data-quality panel, CSV download, and print-to-PDF.

5. Deployment history

The deployment was done in the Render dashboard using the Blueprint file `render.yaml`. The issues found along the way are recorded here because a future redeployment may hit the same ones.

Issue	Cause	Resolution
Blueprint rejected the static site	`plan: free` is not valid for static sites	Removed the line from `render.yaml`.
API build succeeded, migrations failed with error 1105 “insecure transport are prohibited”	An empty `ssl` dictionary is falsy in PyMySQL, so TLS was never enabled	Settings now pass a real TLS context that verifies certificates (`DB_SSL=1`).
Frontend called an unrelated host	Predicted service URL did not match the real one (Render added a suffix)	Set `VITE_API_BASE` to the real API URL and redeployed.
Browser blocked API responses	`CORS_ALLOWED_ORIGINS` did not include the frontend origin	Set it to the frontend URL.

6. Live verification

Check	Result
API health endpoint	HTTP 200, `{"status":"ok"}`
City search on the live site (“Indianapolis”)	Suggestions returned and shown
Live heatwave analysis, Muscat (23.61, 58.54), 1990–2020	106 events; change +638% (1990–94 vs 2016–20); trend +2.97 per decade; Mann–Kendall p = 0.011. Matches the local result.
Automated backend tests (local)	21 tests passing
Production frontend build	Completes without errors

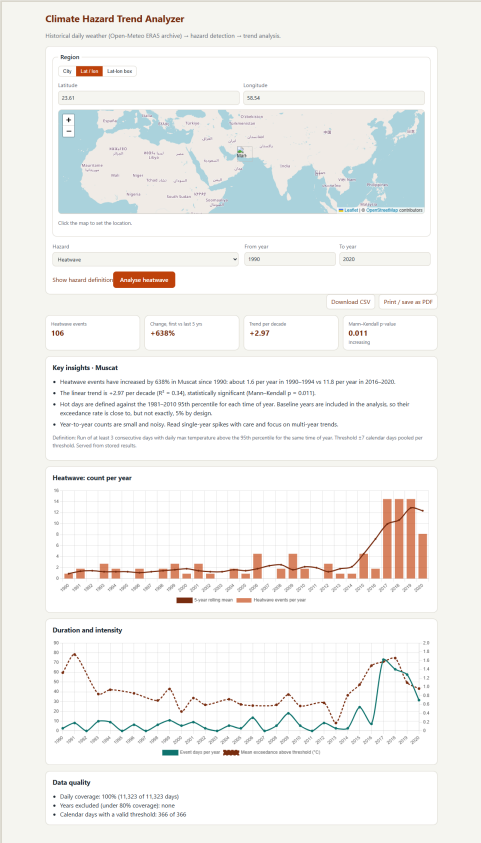


Figure 1. Live analysis on the deployed frontend, Muscat heatwave 1990–2020.

7. Screenshots from setup

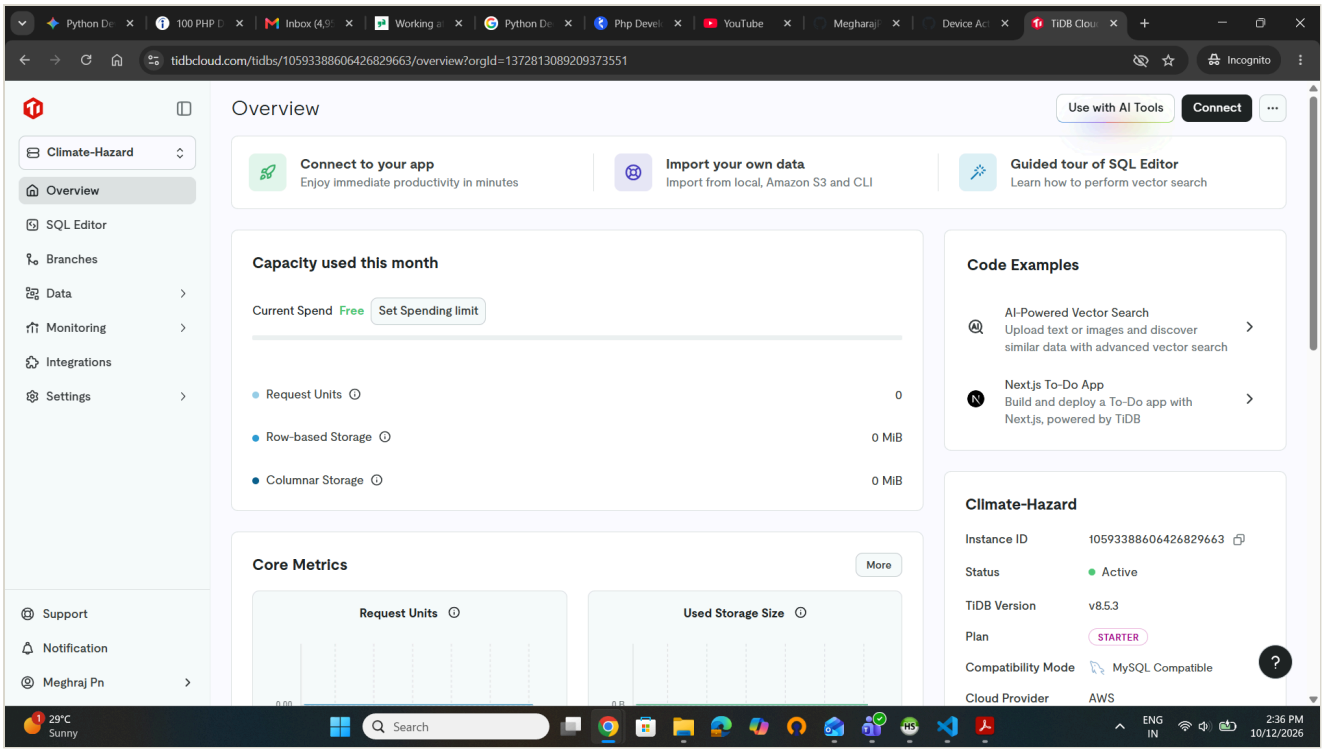


Figure 2. TiDB Cloud cluster "Climate-Hazard": status Active, MySQL compatible, AWS.

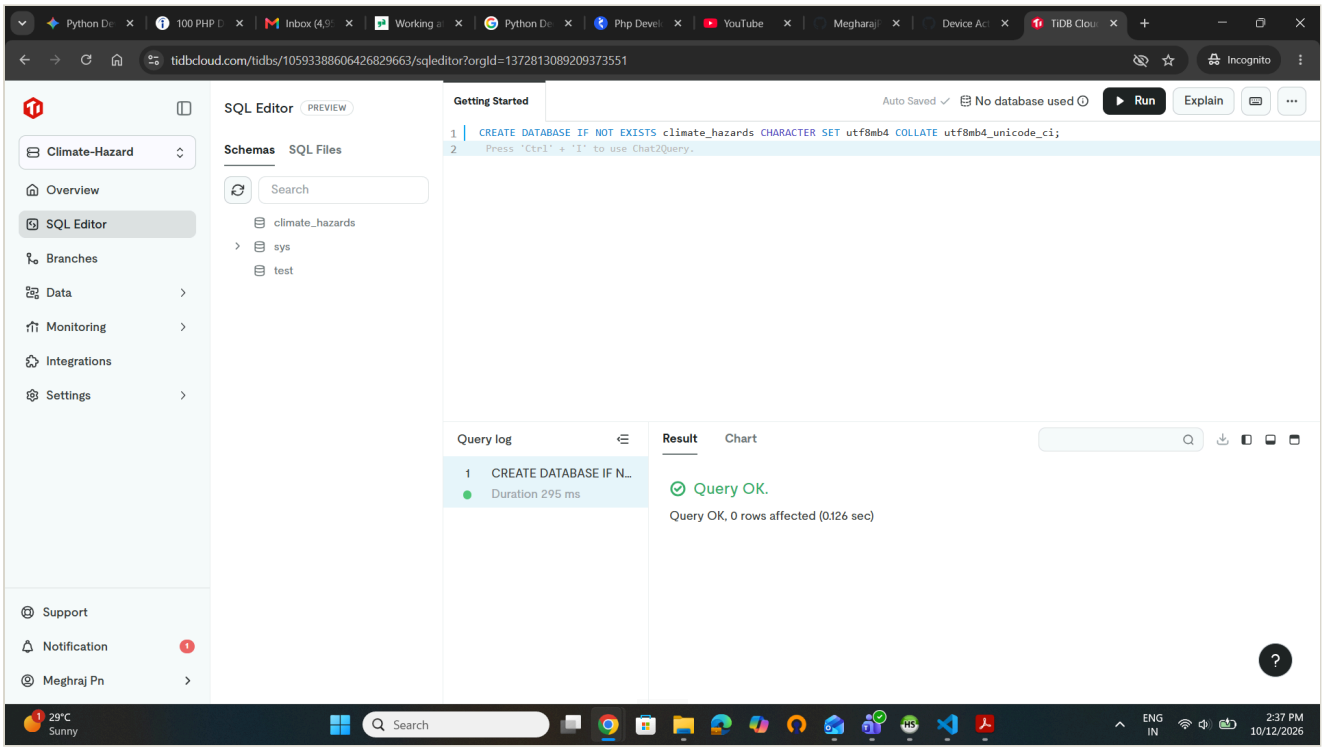


Figure 3. Database `climate_hazards` created in the TiDB SQL editor.

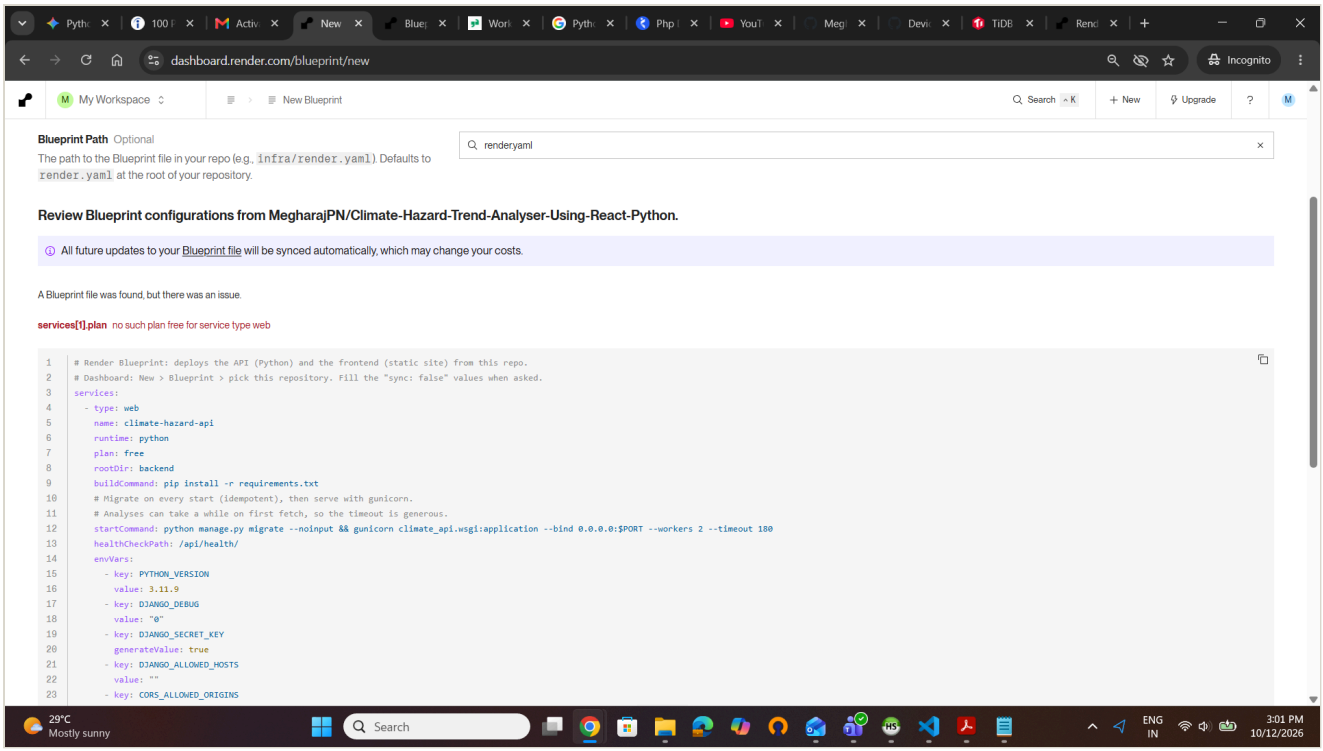


Figure 4. Render Blueprint review of `render.yaml` from the GitHub repository.

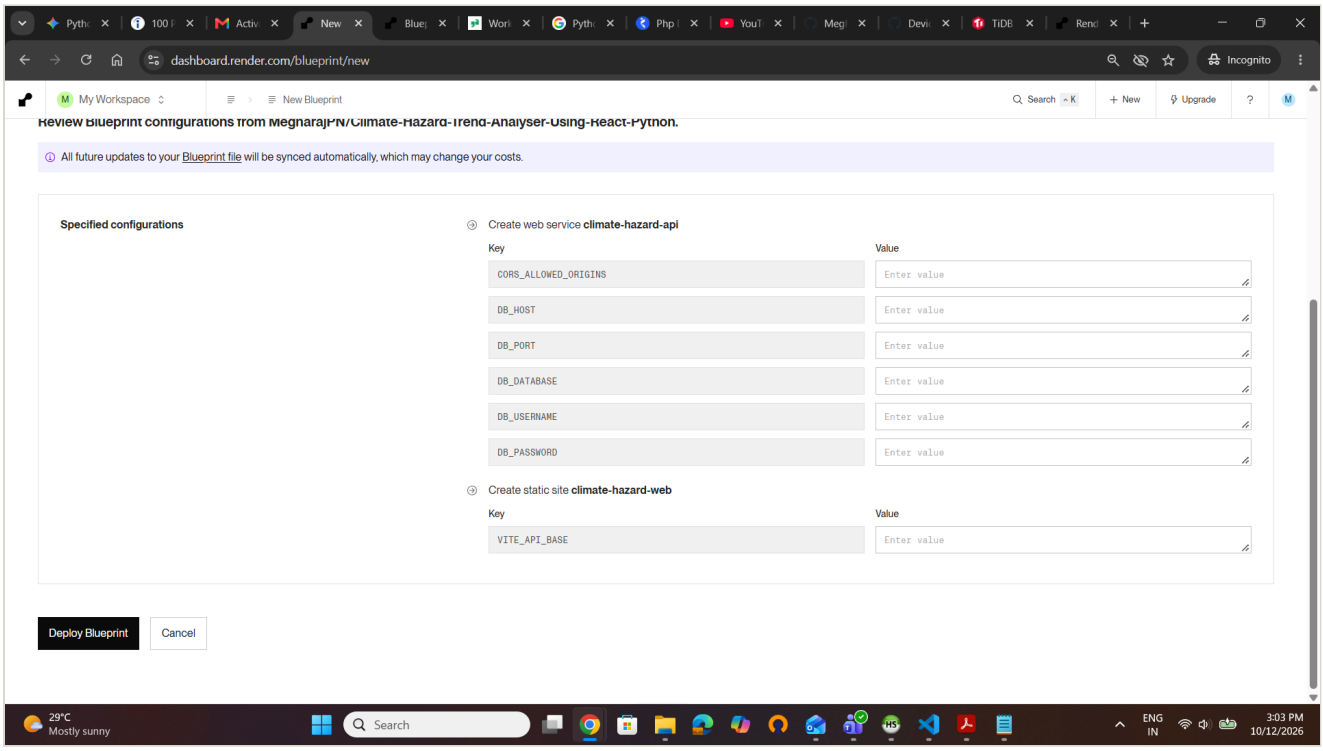


Figure 5. Blueprint configuration form. Secret values were entered by the owner and are not shown.

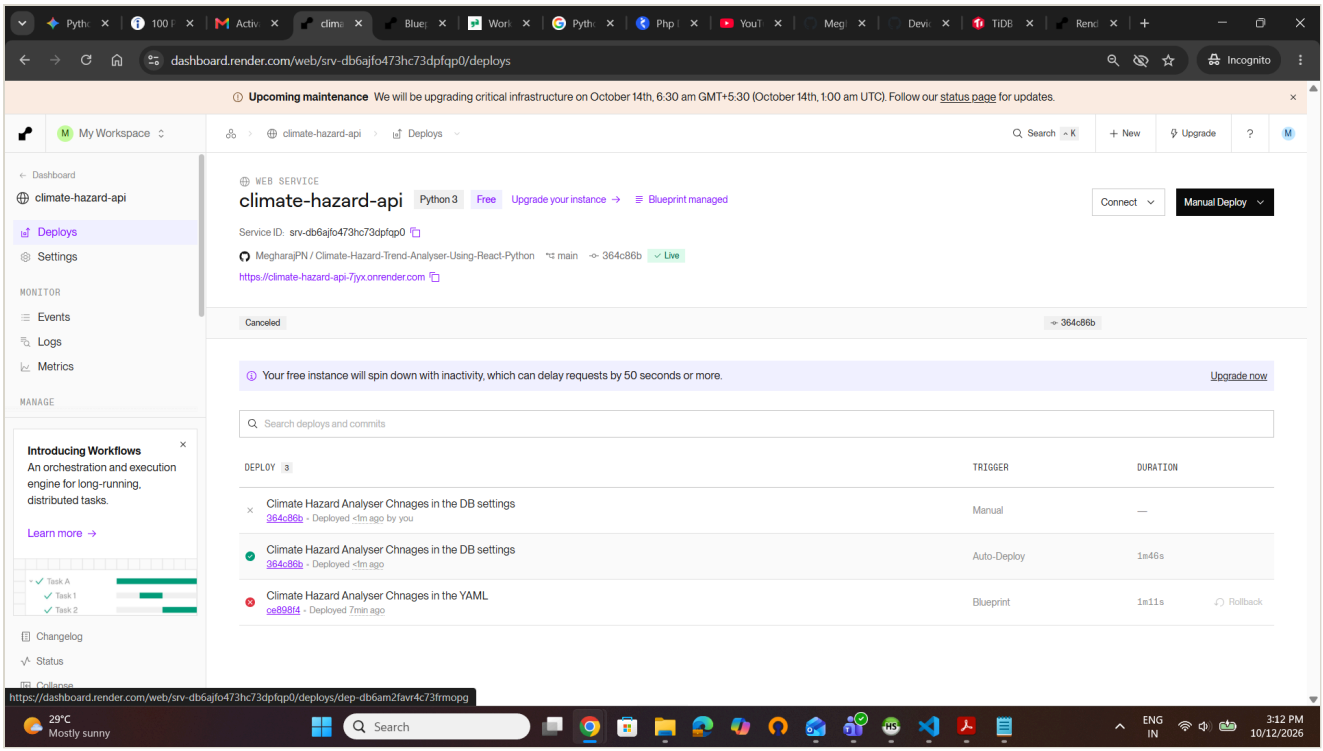


Figure 6. Render deploy history for the API: latest deploy Live, triggered by a push to `main`.

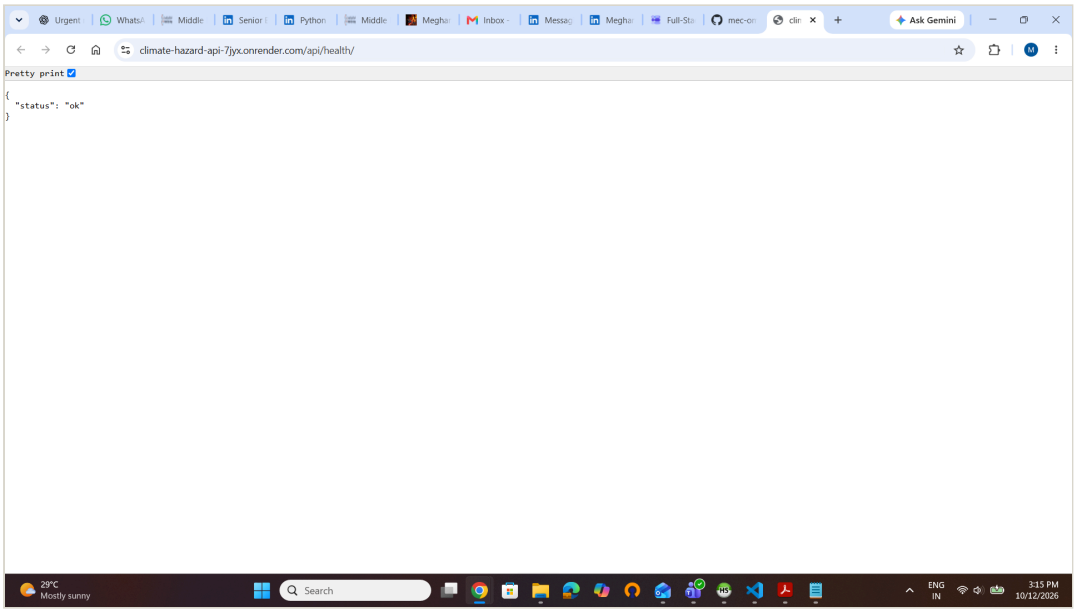


Figure 7. API health check on the live service.

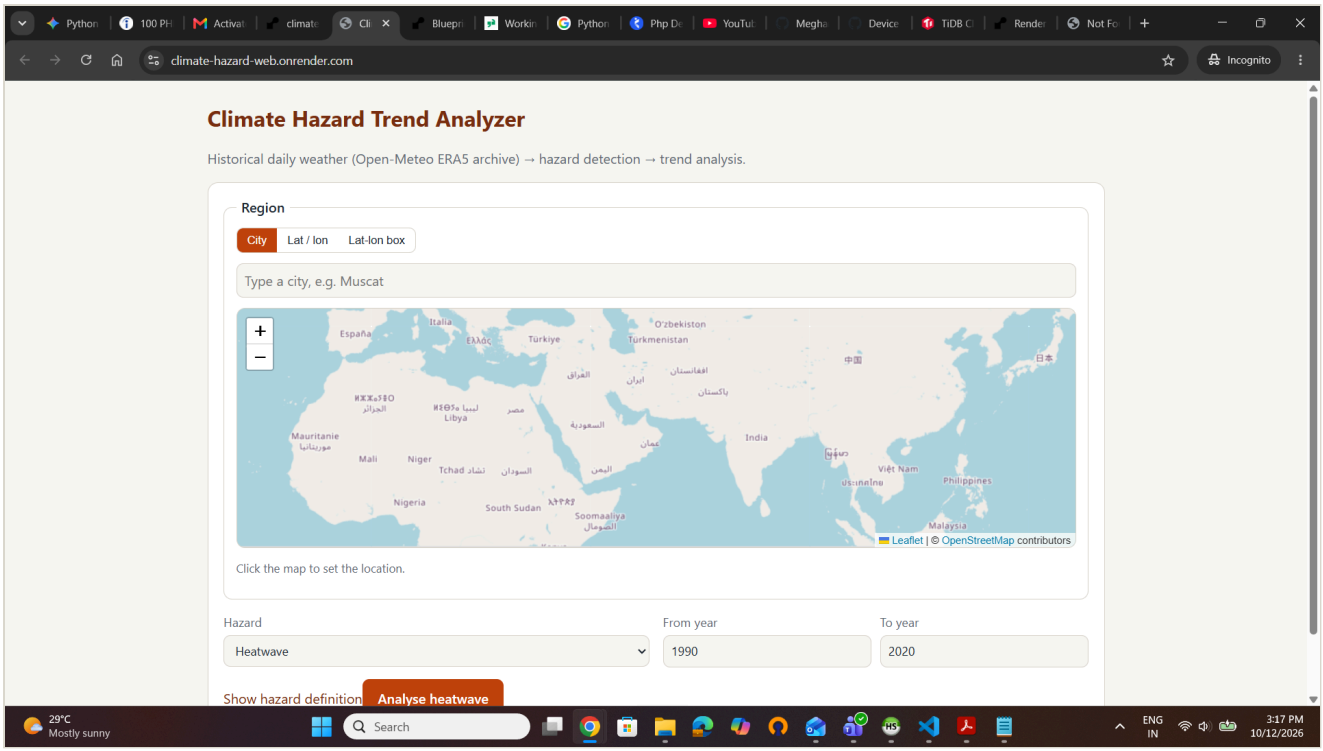


Figure 8. Live frontend at climate-hazard-web.onrender.com.

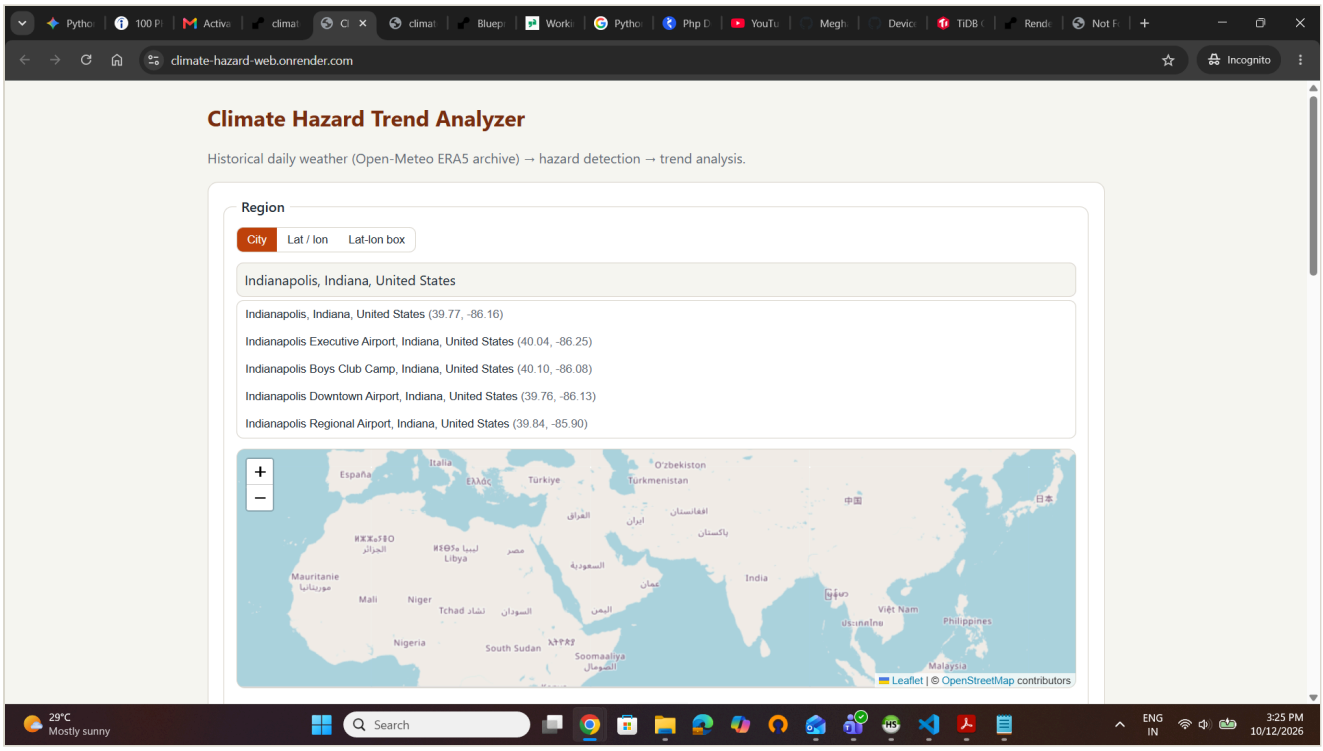


Figure 9. City search working on the live frontend, calling the live API.

8. Operating the system

Routine checks

- Open `https://climate-hazard-api-7jyx.onrender.com/api/health/`. It should return `{"status":"ok"}`.
- Open the frontend, run the Muscat heatwave check, and confirm 106 events.

Redeploying after code changes

- Push to `main`. The API redeploys automatically from the Blueprint.
- The static frontend also redeploys on push. If `VITE_API_BASE` is changed, use **Manual Deploy > Deploy latest commit**.

Changing environment variables

- Edit them in the service's **Environment** tab. The API restarts after a save.
- Changing a variable does not change the code. Check the logs after each change.

9. Troubleshooting

Symptom	Likely cause	Action
First request takes 30–60 seconds	Free service woke from sleep after about 15 minutes idle	Expected. Open the health check first.
"Could not reach the backend" on the frontend	`VITE_API_BASE` is wrong or the API is asleep	Check the variable value, then the health check.
CORS error in the browser console	`CORS_ALLOWED_ORIGINS` does not match the frontend URL	Set it exactly to `https://climate-hazard-web.onrender.com`.
HTTP 502 with "Open-Meteo request failed"	Open-Meteo rate limit or outage	Wait and retry. Cached regions are unaffected.
HTTP 429 from the API	Request limit reached (20 analyses or 60 general requests per hour)	Wait for the window to reset, or raise `RATE_LIMIT_ANALYSIS`.
Migration or TLS error at start	Database credentials wrong, or `DB_SSL` not set to `1`	Check the database variables in the API service, then the TiDB cluster status.

10. Known limitations and open items

- **Map marker icon returns 404.** Leaflet's default marker image is not bundled by the build. The analysis still works; the marker may appear broken. A fix is to import the marker images explicitly in the map component.
- **Unexpected service at the old name.** `https://climate-hazard-api.onrender.com` responds with a redirect (HTTP 307), not this application. It is likely an earlier service from the first deployment attempt. Check its owner in the Render dashboard before deleting it.
- **Free-tier limits.** Render sleeps idle services; TiDB Starter has usage quotas; Open-Meteo applies rate limits.
- **Rate limiter and cache are per process.** With more than one gunicorn worker, limits are counted per worker. Redis would be needed for shared limits.
- **Box analysis** samples nine points, not a full spatial average.
- **Percentage change is sensitive to small counts.** Read it with the regression and Mann–Kendall results.
- **Drought intensity** is not defined; only duration is reported.

- **No automated tests for the frontend** and no CI pipeline yet.

11. Security actions

> - **Rotate the TiDB password.** The generated password appeared in a screenshot during setup. In the TiDB console, reset the password for the user, then update `DB_PASSWORD` in the Render API service and let it redeploy.
> - Keep credentials only in Render and TiDB. Do not commit `.env` files; the repository ignores them.
> - Confirm `DJANGO_DEBUG` is `0` in production and that the Render-generated secret key is in use.

12. Repository layout

```
Climate-Hazard-Trend-Analyser-Using-React-Python/
  render.yaml                 Render Blueprint (API + static site)
  README.md                   Setup, architecture, assumptions, deployment guide
  backend/
    manage.py
    requirements.txt          Pinned Python dependencies (includes gunicorn)
    .env.example              Template for local configuration (no secrets)
    climate_api/              Django project: settings, URLs, WSGI
    hazards/                  App: ingest, detection, heatwave/rain/drought rules, trends,
                              analysis orchestration, API views, throttling, tests
  frontend/
    package.json, vite.config.js, index.html
    src/                      React app: App, RegionPicker, HazardCharts, api client, styles
```

13. Local development

```
# Backend
cd backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env          # set local database values and a secret key
python manage.py migrate
python manage.py test hazards
python manage.py runserver 8000

# Frontend
cd frontend
npm install
npm run dev                     # http://localhost:5173 (proxies /api to port 8000)
```

14. AI tools used

Claude Code (Anthropic) was used to scaffold the project, write the hazard logic, tests, and interface, and to assist with deployment configuration and troubleshooting. The project owner reviewed the code, ran

the tests and live checks above, and made the definitional choices described in the implementation report.